Forecasting Next-Day Household Electricity Use in London With Recurrent Neural Networks

Patrick Gilliland

Udacity

AI Mastery Capstone: Deep Learning Systems

Udacity mentor

October 16, 2026

Forecasting Next-Day Household Electricity Use in London With Recurrent Neural Networks

Report Overview

This report addresses forecasting one London household's next 24 hours of electricity use, 48 half-hourly readings in kilowatt-hours (kWh), from the 336 readings of its previous seven days. The data are UK Power Networks' Low Carbon London smart meter readings (UK Power Networks, 2015b), from which I used 1,000 households with every reading of 2013. This is a sequential or time-series task using a Recurrent Neural Network (RNN), with a long short-term memory (LSTM) network as the baseline model and a gated recurrent unit (GRU) network as the experimental model, differing only in the layer type. On the test months the GRU had the lower mean absolute error (MAE), and both models beat three naive forecasts on MAE, though the best naive forecast put the day's peak in the right 4-hour block more often.

Dataset and Task Description

The data hold the half-hourly electricity readings of London households in UK Power Networks' Low Carbon London trial. The publisher describes the data set as holding "energy consumption, in kWh (per half hour), unique household identifier, date and time" (UK Power Networks, 2015b, para. 3), and each row also carries the household's tariff group. So one reading is one household's use in one half-hour. The households were supplied by EDF Energy, and the trial collected readings for "the full 2013 calendar year" (UK Power Networks, 2015a, p. 12). Part of the sample was on a time-of-use tariff, priced high, low or normal by signals "given a day ahead" (UK Power Networks, 2015b, para. 4). I call the two groups the standard tariff group and the time-of-use group.

The download holds 167,932,474 rows from 5,566 household identifiers, one fewer than the 5,567 households the publisher describes (UK Power Networks, 2015b). After cleaning, 93,024,217 readings fall in 2013, from 5,528 households. Of these, 1,132 have every one of the year's 17,520 half-hours: 881 in the standard tariff group and 251 in the time-of-use group. I drew 1,000 of them at random with seed 42, giving 781 and 219. The working file holds 365,000 household-days and 17,520,000 readings.

The data suit the task. Each household's readings form an ordered series at a fixed step, and an RNN is a network "specialized for processing a sequence of values" (Goodfellow et al., 2016, p. 367). Use has daily and weekly cycles (UK Power Networks, 2015a, p. 20; Haben et al., 2021, sec. 2.1), so a week of input holds both. A single household is a hard target, because the load of small groups of households is irregular and volatile (Haben et al., 2021, sec. 1.1). The task matters because next-day forecasts of residential load can drive a battery's charge and discharge schedule (Haben et al., 2021, sec. 5.2.1). Haben et al. (2021, sec. 6.1) also name this trial's data as a large, recent data set for testing such models.

My pipeline prepared the data in the five stages below. Its manifest counts the same work in seven numbered steps.

1. It dropped the 5,560 "Null" readings, which hold no value.
2. It kept one of each exact duplicate (115,453 dropped), which loses nothing.
3. It kept the households with every 2013 reading, so no reading was filled in. It drew the sample with seed 42, so the subset can be rebuilt.
4. It rounded every reading to the nearest third decimal (the manifest's step 7). Rounding up would have raised every reading it touched. The publisher's file holds 1,015,466 readings

with more than three decimals, such as 1.4529999, and rounding changed 115,323 sample readings by at most 0.0000005 kWh.

5. It kept the time stamps as published. They behave as Greenwich Mean Time (GMT) all year: averaged over all households, the weekday morning rise is half-way up at 07:00 before the clocks went forward on 2013-03-31 and at 06:00 after. Both change dates, 2013-03-31 and 2013-10-27, are last Sundays, which fits the UK rule as GOV.UK (2025) gives it today.

The split is by date, so a forecast never sees its own future. Forecast accuracy can only be judged on data not used to fit the model (Hyndman & Athanasopoulos, 2021, sec. 5.8), so the test months play no part in fitting. The training months are 1 Jan to 31 Aug 2013, the validation months 1 Sep to 31 Oct and the test months 1 Nov to 31 Dec. Each window is one household and one target day: the 336 readings of the week before as input, and the day's 48 readings as target. The first target day is 2013-01-08, so no input reaches before 1 Jan. That gives 236,000 training windows and 61,000 each for validation and test. One scaler, the mean (0.2158 kWh) and standard deviation (0.3112 kWh) of the training months' readings, applies to inputs and targets. For the same reason it uses no reading from later months (Hyndman & Athanasopoulos, 2021, sec. 5.8). Beside the models I score three naive forecasts: the same half-hour yesterday, the same half-hour last week, and the mean of the previous 7 days at each half-hour.

Before any model trained, I listed every anomaly, wrote each rule from the training and validation months only, and kept every row. Van den Broeck et al. (2005, p. 968) limit the options for a suspect value to correcting, deleting or leaving it unchanged. I left each unchanged and flagged it. I report results with and without the flagged households, as they advise for outliers that remain (Van den Broeck et al., 2005, p. 969). In all, 74 households carry at least one

of five flags in the training or validation months. The flags mark a day or more of zero readings, a reading above 4.8548 kWh (the 99.99th percentile of those months' readings), or the same non-zero reading for a day or more. They also mark mean use below 0.01 kWh per half-hour, and a month at least 3 times, or at most a third of, the month before. Both models train on every household, because a forecaster for real households has to cope with unusual ones.

Model Architecture and Design Decisions

The baseline model (the LSTM) reads the scaled input week one half-hour at a time, 336 steps of one feature. Two stacked LSTM layers of 64 hidden units carry a state from step to step. A linear head maps the top layer's last state, 64 values, to the 48 half-hours of the target day. The state starts at zero for every window, because I treat each window as a separate sample and carry nothing between them. The model has 53,552 trainable parameters, 50,432 in the recurrent layers and 3,120 in the head. I built both models in PyTorch, a Python deep learning library. The comparison follows the course's own exercise comparing LSTM and GRU layers, but here both models share one set of settings and three seeds, use the L1 loss on readings in place of cross-entropy, split the data by date and train in float32.

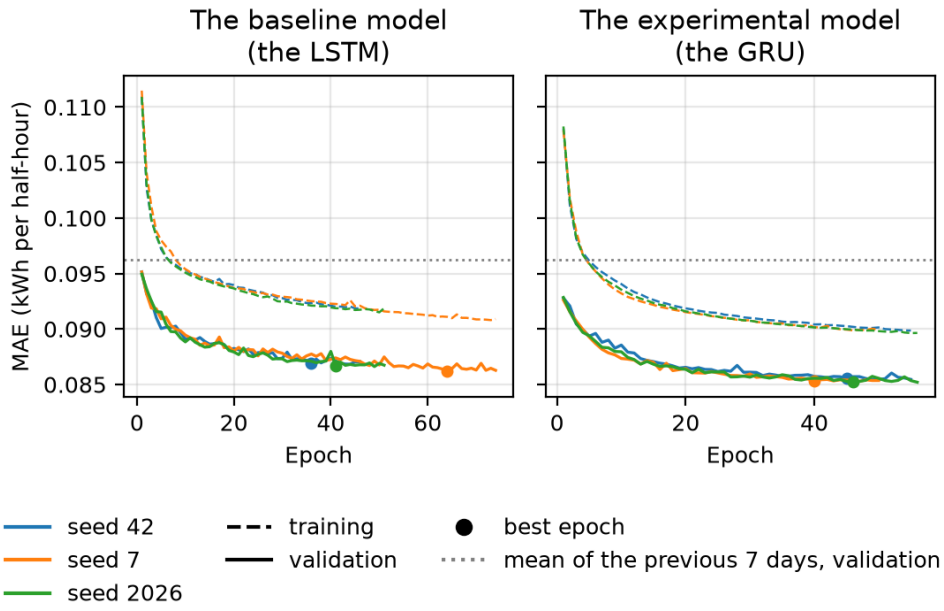
Layer Types. An RNN "shares the same weights across several time steps" (Goodfellow et al., 2016, p. 368), so one set of weights reads the whole week. A plain RNN struggles over long sequences, because gradients passed back over many steps tend to vanish or explode (Goodfellow et al., 2016, sec. 10.7). The LSTM's memory cell, its "constant error carrousel", lets error flow back across long gaps (Hochreiter & Schmidhuber, 1997, sec. 3.2). Gated networks are the usual default for sequence input (Goodfellow et al., 2016, sec. 11.2). LSTM networks were also the first popular deep learning choice for household and other low voltage load (Haben et al., 2021, sec. 2.3.5). My model runs the LSTM as PyTorch defines it, with input, forget, cell

and output gates (PyTorch, n.d.-c). In this run every model beat the validation MAE of the mean of the previous 7 days from its first epoch.

Depth. Two stacked layers let the lower layer turn the raw readings into a representation the upper layer uses (Goodfellow et al., 2016, sec. 10.5). The same section notes that, in general, "it is easier to optimize shallower architectures" (Goodfellow et al., 2016, p. 394), so I kept to two. I kept the width small and the same in both models, so only the layer type changes, and Chung et al. (2014, p. 6) also made their models small to avoid overfitting, which "can easily distract the comparison". The training curves in Figure 1 bear this out: in every run validation MAE flattens without a sustained rise, so two layers of 64 showed no sign of overfitting on the validation months. The depth and the width were set at the outset and not tuned, a limitation I return to below.

Figure 1

Training and Validation Mean Absolute Error by Epoch for Each Model and Seed



Note. Training MAE (dashed) is each epoch's L1 loss times 0.3112 kWh. A dot marks each run's best epoch, and the dotted line is the validation MAE of the mean of the previous 7 days.

Activations. Each LSTM layer uses the sigmoid function in its gates and the hyperbolic tangent on its cell's input and output (PyTorch, n.d.-c). The head has no activation. The scaled targets go below zero, so a sigmoid or hyperbolic tangent would cap the forecast, and a rectified linear unit would cut off every value below the mean. The cost is that a forecast can fall below zero kWh. I decided before scoring either model on the test months that such forecasts are scored as they are, since clipping would change the MAE. On the test months 4,506 to 13,682 of each run's 2,928,000 forecast half-hours were negative, the lowest -0.3553 kWh.

Output Format. The head forecasts all 48 half-hours at once, a direct forecast, so no forecast feeds the next. Haben et al. (2021, sec. 6.1) ask every study to say whether it forecasts directly or iteratively. Reading only the last state summarises the week into "a fixed-size representation used as input for further processing" (Goodfellow et al., 2016, p. 376). Undoing the scaler turns the forecasts back into kWh, so the MAE is in kWh per half-hour. On the test months both models' error was largest at 18:30, in the early evening, and not at the end of the day.

Training Setup. Both models share these settings, fixed before any run:

- The L1 loss, the mean absolute error between forecast and target (PyTorch, n.d.-b). With one linear scale for every reading, the L1 loss on scaled values is the MAE divided by the standard deviation, so loss and metric rank forecasts alike. Error metrics should follow from the problem being solved (Goodfellow et al., 2016, sec. 11).
- Adam at a learning rate of 0.001, its authors' default for the problems they tested (Kingma & Ba, 2015, p. 2).
- Batches of 256 windows, the top of the typical range of power of 2 sizes (Goodfellow et al., 2016, sec. 8.1.3).

- Gradient-norm clipping at 1.0, which bounds the step when a gradient explodes (Goodfellow et al., 2016, sec. 10.11.1), as Chung et al. (2014, sec. 4.2) did in their LSTM and GRU comparison.
- Early stopping after 10 epochs in a row without a fall in validation MAE of more than 0.0001 kWh, or at 100 epochs. The run keeps the best epoch: the last epoch that lowered the best validation MAE so far by more than that minimum. Keeping the weights of the best validation epoch is standard early stopping (Goodfellow et al., 2016, sec. 7.8; Prechelt, 1998, sec. 2.1). Because of my margin, in five of the six runs a later epoch scored lower, by at most 0.00009 kWh. I chose a patient rule because, on the one data set Prechelt (1998, sec. 3.4) split in time order, one family of his stopping criteria failed badly while the slower variants of the other two held up well.
- Three seeds per model, 42, 7 and 2026, because "the initial point can affect the generalization" (Goodfellow et al., 2016, p. 297).
- Float32 throughout, because bfloat16 was no faster at this size on this rig's graphics processing unit (GPU). Deterministic kernels, because PyTorch warns of non-determinism in recurrent layers on some library versions (PyTorch, n.d.-c).

Figure 1 shows how these choices played out. A one-epoch pre-flight on seed 42 first gave a finite loss that fell within the epoch, from 0.4299 over the first 92 batches to 0.3393 over the last 92. Every full run then beat the validation MAE of the mean of the previous 7 days, 0.0962 kWh, from its first epoch, and each model's three seeds track one another closely. The runs stopped after 46 to 74 epochs and kept best epochs 36 to 64, so the 100-epoch ceiling never bound. Overfitting shows as a gap between training and test error that grows too large (Goodfellow et al., 2016, sec. 5.2). Here validation MAE flattens without a sustained rise, and

training MAE stays above it in every run. That is consistent with the training months including the winter, when use is higher (0.2158 kWh per half-hour over the training months against 0.1955 on the validation months). Training MAE is also averaged while the weights still change. On LSTM seed 42, epoch 44 reached 0.08683 kWh but beat epoch 36's 0.08692 by less than 0.0001 kWh, so epoch 36 stayed the best epoch.

Experimental Comparison

The one change is the layer type (LSTM to GRU). The experimental model (the GRU) is the same model code with PyTorch's GRU layer in place of its LSTM layer, and nothing else changes (Table 1). The GRU was proposed as a hidden unit "motivated by the LSTM unit but is much simpler to compute and implement", with a reset gate, an update gate and no separate memory cell (Cho et al., 2014, p. 1726). In it, "a single gating unit simultaneously controls the forgetting factor and the decision to update the state unit" (Goodfellow et al., 2016, p. 407). PyTorch's GRU departs slightly from the paper's equations, on purpose (PyTorch, n.d.-a), and my code runs PyTorch's form.

The comparison is meaningful because it answers an open question with everything else held fixed. On polyphonic music and speech signal modelling the GRU was "comparable to LSTM" (Chung et al., 2014, p. 1). Later studies found no variant of either that clearly beat both across a wide range of tasks (Goodfellow et al., 2016, sec. 10.10.2). Chung et al. (2014, p. 7) concluded that the choice "may depend heavily on the dataset and corresponding task", so the answer for household load is open. It bears on cost, because the GRU has fewer parameters than the LSTM (Haben et al., 2021, sec. 2.3.5). Changing one thing makes this the controlled comparison that comparing two learning algorithms requires (Goodfellow et al., 2016, sec. 7.4). Both models see the same data, split, windows, scaler, settings, seeds and stopping rule.

Table 1

Settings of the Baseline Model and the Experimental Model

Setting	The baseline model (the LSTM)	The experimental model (the GRU)
Layer type	`nn.LSTM`	`nn.GRU`
Trainable parameters	53,552	40,944
Initialisation	PyTorch's default, with no change to the forget-gate bias	PyTorch's default
Hidden units	64 in each layer	The same
Layers	2	The same
Head	Linear, the last step's 64 values to 48 half-hours	The same
Loss	L1 on scaled values	The same
Optimiser	Adam, learning rate 0.001	The same
Batch	256 windows, shuffled by a generator seeded with the run's seed	The same
Gradient-norm clipping	At 1.0	The same
Stopping rule	10 epochs without a gain of more than 0.0001 kWh of validation MAE, at most 100 epochs, best epoch restored	The same
Seeds	42, 7 and 2026	The same
Scaler	One mean and standard deviation from the training months	The same
Windows	336 readings in, 48 out	The same
Split	The training, validation and test months, by date	The same
Households	All 1,000, the flagged ones included	The same
Precision	Float32, deterministic kernels	The same

Note. The parameter count follows from the layer type, so it is part of the one change, not a second one. LSTM = long short-term memory; GRU = gated recurrent unit; MAE = mean absolute error.

The parameter count follows from the layer type, so it is part of the one change. Each LSTM layer has four gate blocks where the GRU's has three (PyTorch, n.d.-a, n.d.-c). The GRU has 40,944 trainable parameters against the LSTM's 53,552: 12,608 fewer, so the LSTM has 31% more. Chung et al. (2014, p. 6) matched parameter counts instead, but that changes the width as well. I kept the width fixed and name the gap.

Two rules were written before either model was scored on the test months. Under the comparison rule, a gap in mean MAE smaller than the larger of the two models' standard deviations over their seeds counts as no clear difference. I applied it to the validation months first. Where a view needs one set of forecasts per model, it uses the seed with the median validation MAE, fixed as LSTM seed 2026 and GRU seed 7. Such a rule is needed because one seed's run can be a lucky draw, as "the initial point can affect the generalization" (Goodfellow et al., 2016, p. 297). I re-ran all the code several times: in a fresh install of the listed packages, with older releases of pandas and PyTorch, and after later changes to the figures and to the checks on the code's inputs. The code that forecasts the test months did not change, and each time the test-month forecasts and the trained weights came out identical. While preparing the data I also scored the mean of the previous 7 days on the test months, but no choice used that score.

Results and Interpretation

The three naive forecasts set the level to beat, as a new method is "not worth considering" if it cannot beat simple ones (Hyndman & Athanasopoulos, 2021, sec. 5.2). The same half-hour yesterday and last week are seasonal naive forecasts (Hyndman & Athanasopoulos, 2021, sec. 5.2). The mean of the previous 7 days is a simple average, a model Haben et al. (2021, sec. 2.2.1) treat as a serious benchmark. I chose MAE as the lead metric before any model trained, because every household is scored in kWh. Hyndman and Koehler (2006) prefer the MAE where all

series share a scale, since it "is simpler to explain" (sec. 5), and the mean absolute scaled error (MASE) where scales differ and data sit near zero, as here. So the MASE per household sits beside the MAE. It divides each household's MAE by that of the same half-hour yesterday over its training months (Hyndman & Koehler, 2006, sec. 3; Hyndman & Athanasopoulos, 2021, sec. 5.8). Root mean squared error (RMSE), which is more sensitive to large errors (Hyndman & Koehler, 2006, sec. 2.1), is reported too. Percentage errors are out, because the data hold exact zeros, where they are "infinite or undefined" (Hyndman & Koehler, 2006, sec. 2.2). Table 2 gives the scores on the test months.

On the test months the experimental model (the GRU) has an MAE of 0.1067 kWh per half-hour and the baseline model (the LSTM) 0.1085. The best naive forecast, the mean of the previous 7 days, scores 0.1176. Without the 74 flagged households the GRU scores 0.1014, the LSTM 0.1032 and the mean of the previous 7 days 0.1125, so the ranking does not rest on them. The median MASE per household, 0.8368 for the GRU and 0.8527 for the LSTM, is below 1. So on at least half of the households both models beat the error the same half-hour yesterday made on that household's training months. By tariff group, both models beat the naive forecasts in both groups. The time-of-use group's MAE divided by mean use is higher for every forecast (for the GRU 0.4465 against 0.4257). That group also uses less (0.2028 kWh per half-hour against 0.2614), so I do not read the gap as an effect of the price signals.

Table 2

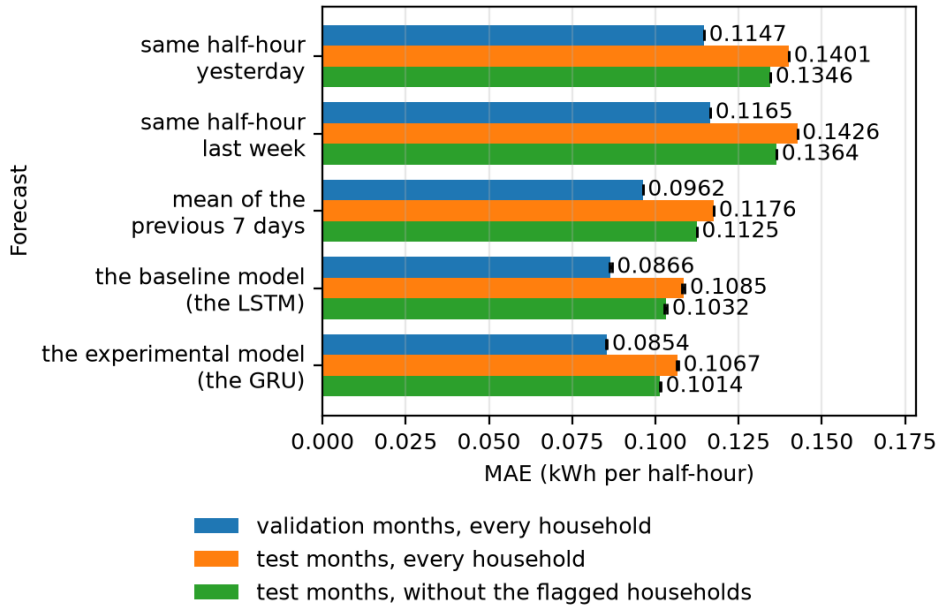
Test Month Scores of the Two Models and the Three Naive Forecasts, With and Without the Flagged Households

Forecast	MAE (kWh)	MAE / mean use	MASE, mean	MASE, median	RMSE (kWh)
Every household					
Same half-hour yesterday	0.1401	0.5639	1.1492	1.1143	0.2914
Same half-hour last week	0.1426	0.5739	1.1709	1.1216	0.2939
Mean of the previous 7 days	0.1176	0.4731	0.9647	0.9199	0.2299
The baseline model (the LSTM)	0.1085 ± 0.0004	0.4365 ± 0.0016	0.8995 ± 0.0078	0.8527 ± 0.0048	0.2366 ± 0.0008
The experimental model (the GRU)	0.1067 ± 0.0002	0.4294 ± 0.0009	0.8909 ± 0.0048	0.8368 ± 0.0006	0.2326 ± 0.0011
Without the 74 flagged households					
Same half-hour yesterday	0.1346	0.5701	1.1429	1.1128	0.2737
Same half-hour last week	0.1364	0.5779	1.1596	1.1166	0.2751
Mean of the previous 7 days	0.1125	0.4763	0.9555	0.9129	0.2147
The baseline model (the LSTM)	0.1032 ± 0.0004	0.4369 ± 0.0017	0.8727 ± 0.0040	0.8452 ± 0.0048	0.2203 ± 0.0007
The experimental model (the GRU)	0.1014 ± 0.0002	0.4296 ± 0.0008	0.8576 ± 0.0019	0.8301 ± 0.0016	0.2168 ± 0.0010

Note. The test months are 1 Nov to 31 Dec 2013. Errors are in kWh per half-hour. Figure 2 sets the MAE values beside those of the validation months. For the two models each value is the mean over seeds 42, 7 and 2026, with the standard deviation over the seeds after the ± sign. MAE = mean absolute error; MASE = mean absolute scaled error; RMSE = root mean squared error; LSTM = long short-term memory; GRU = gated recurrent unit.

Figure 2

Mean Absolute Error of Every Forecast on the Validation and Test Months



Note. Model bars are the mean over three seeds, with one standard deviation as a line. The third bar of each forecast leaves out the 74 flagged households.

Under the comparison rule the GRU is lower. On the validation months its mean MAE, 0.08537 kWh, is 0.00124 below the LSTM's 0.08661, against a larger seed standard deviation of 0.00037. On the test months the gap is 0.00178 against 0.00039. As an extra view I paired the models by household. The LSTM's MAE exceeds the GRU's by 0.00178 kWh on average, and the GRU is lower for 872 of the 1,000 households. I resampled the 1,000 households with replacement 10,000 times, each keeping its two errors together (Efron, 1979, p. 3; James et al., 2023, sec. 5.2). Taking the 2.5th and 97.5th percentiles of the resampled means, my own choice, gives a 95% bootstrap interval of 0.00158 to 0.00198 kWh. A comparison needs that statement of its uncertainty, because a test set leaves "statistical uncertainty around the estimated average test error" (Goodfellow et al., 2016, p. 120). Figure 2 shows every forecast erring more on the

test months than on the validation months, which fits the higher use in November and December (0.2485 kWh per half-hour on the test months against 0.1955 on the validation months). MAE divided by mean use moves much less: for the GRU, 0.4294 on the test months against 0.4366 on validation.

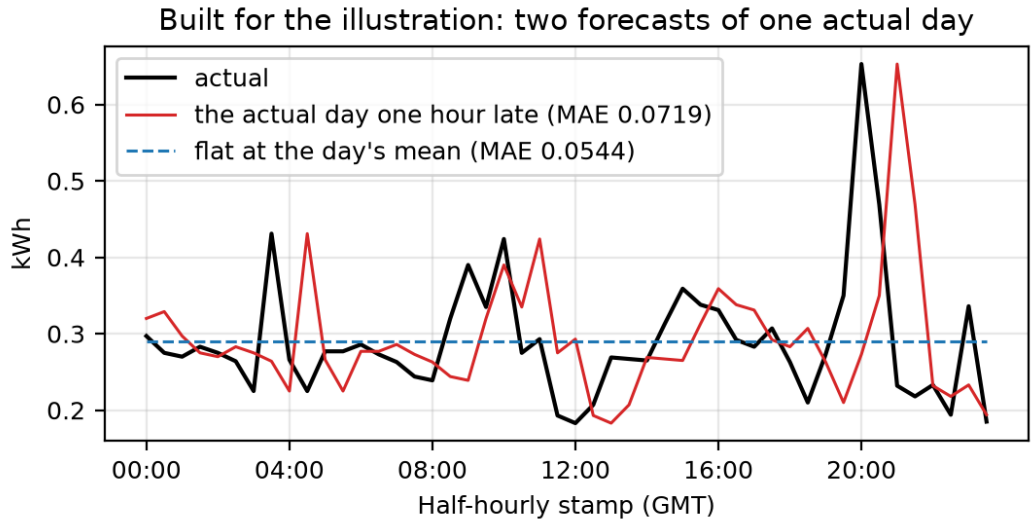
The MAE lead has two costs. The first is on RMSE, where the LSTM scores 0.2366 kWh and the GRU 0.2326, both above the 0.2299 of the mean of the previous 7 days. This is consistent with the loss: "A forecast method that minimises the MAE will lead to forecasts of the median, while minimising the RMSE will lead to forecasts of the mean" (Hyndman & Athanasopoulos, 2021, sec. 5.8). Readings are skewed: mean use over the training months is 0.2158 kWh per half-hour, yet the highest training or validation reading is 8.2040 kWh. So a median forecast sits low and pays most on the large errors RMSE weighs. Bias shows it. Forecasts whose errors do not average zero are biased (Hyndman & Athanasopoulos, 2021, sec. 5.4), and I measure bias as the mean of actual less forecast, divided by mean use. On their median seeds both models forecast low on every kind of price-signal day in the test months (no signal, high, low or both), in both groups, with biases from 17.0% to 22.9% of mean use. The mean of the previous 7 days stays between -2.5% and +2.1%.

The second cost is timing, through the double penalty. MAE scores each half-hour on its own. A forecast with the right peak at the wrong time is penalised twice, once for the peak it forecasts and once for the one it misses, so a flat forecast can beat a better-informed peaky one (Haben et al., 2021, sec. 3.4). Figure 3 shows this on household MAC000604 (the publisher's identifier) on 2013-03-07, a training-month day, with two forecasts built for the illustration, not by a model. A flat forecast at the day's mean scores an MAE of 0.0544 kWh. The actual day moved one hour late scores 0.0719, though it holds every rise and peak of the real day. My

reading is that a model trained on MAE is pushed the same way: when it cannot tell when a peak will come, a smooth forecast costs less.

Figure 3

How the Double Penalty Favours a Flat Forecast on One Household-Day



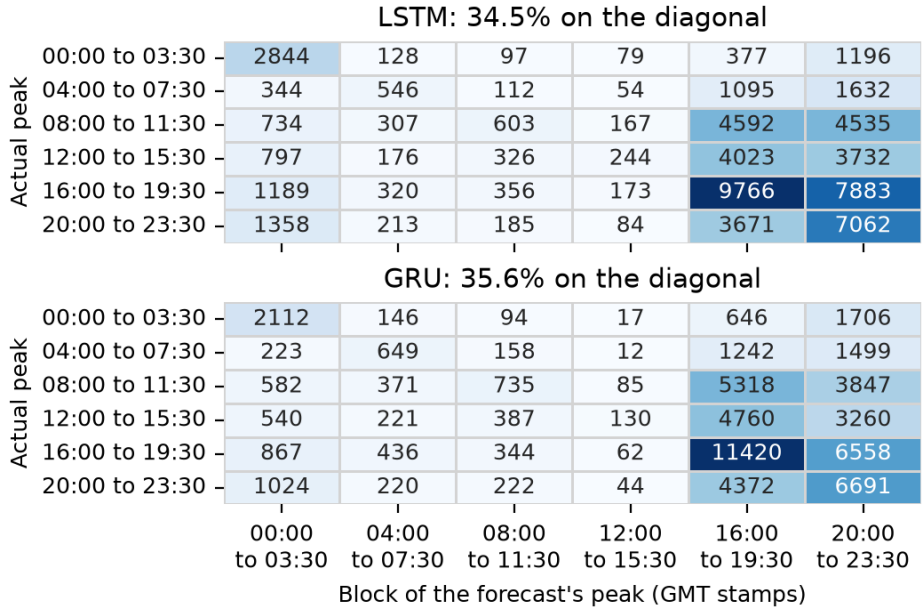
Note. Both forecasts are built for the illustration from MAC000604's readings on 2013-03-07, a day in the training months, and neither comes from a model. MAE is in kWh per half-hour.

The peak-timing confusion matrix, a view derived from the forecasts, measures what MAE cannot (Figure 4). A confusion matrix sets each actual class against the predicted class, so it shows which errors are being made (James et al., 2023, sec. 4.4.2). A forecast of 48 readings has no classes, so I made one fit. For each test household-day, the 4-hour block (six a day) holding the actual peak is set against the block holding the forecast's peak. The LSTM puts the peak in the right block on 34.5% of test household-days and the GRU on 35.6%, both below the 40.3% of the mean of the previous 7 days. Most forecast peaks fall in the two evening blocks, 16:00 to 23:30, whenever the actual peak comes at 04:00 or later (72.1% to 91.3% of household-

days). So the models' lower MAE comes with poorer peak timing, which suggests their gain comes from the level and shape of the day, not from timing each day's peak.

Figure 4

Peak-Timing Confusion Matrix of the Two Models on the Test Months



Note. Household-days counted by the 4-hour block holding the actual peak (rows) against the block holding the forecast's peak (columns), for LSTM seed 2026 and GRU seed 7. Stamps are GMT.

On the median seeds the error is largest at 18:30 for all three forecasts: 0.1710 kWh for the LSTM, 0.1673 for the GRU and 0.1814 for the mean of the previous 7 days. I also sorted each test household-day into the first of five day types that applies: Christmas (24 to 26 Dec), after the clock change (a day whose input week holds the 27 Oct change), a price-signal day for a time-of-use household, weekend and weekday. Christmas has the highest MAE of the five, 0.1234 for the LSTM and 0.1233 for the GRU. On an ordinary weekday, MAC001618 on 2013-11-28, the models follow the daily shape but smooth it, with MAEs of 0.0781 kWh for the

LSTM and 0.0758 for the GRU against 0.0852 for the mean of the previous 7 days. They miss two short evening spikes, as that naive forecast does. The worst household-day of every day type comes from a heavy-use household, and three of the five from MAC004319, one of the flagged households.

The clearest failure case is Christmas Day for MAC004319. It used 99.79 kWh, most of it from 09:00 on, and both models forecast a much quieter day. Their MAEs, 1.5298 kWh for the LSTM and 1.5629 for the GRU, are worse than the 1.3277 of the mean of the previous 7 days. No input marks the day as Christmas, and both models forecast low and smooth, so a heavy day unlike the input week is where they fail most. A training-behaviour example comes from Figure 1: the GRU learns faster. After the first epoch its validation MAE is 0.09261 to 0.09284 kWh across seeds, against 0.09494 to 0.09515 for the LSTM. At every epoch up to the 46th, where the shortest run stops, each GRU seed is below each LSTM seed.

The trade-offs favour the GRU on size and MAE. It reaches the lower MAE with 12,608 fewer parameters, though its lead over the LSTM is small beside either model's lead over the mean of the previous 7 days. Haben et al. (2021, sec. 6.1) ask for running times beside errors. On the GPU, an NVIDIA GeForce RTX 5090, both models trained at about the same speed per epoch (3.7 to 3.8 seconds for the GRU and 3.8 for the LSTM in this run). All six runs with the pre-flight took 21.0 minutes. On the GPU both models forecast a batch of 256 within a few milliseconds, and the times varied too much between runs of the same code to rank the two. On this machine's central processing unit (CPU), an Intel Core Ultra 9 285K, with PyTorch fixed at 16 threads, a median GRU training step took 457 milliseconds against 579 for the LSTM. In this run its CPU forecasting step was the slower, 65 milliseconds against 26. Training all six runs and

the pre-flight on that CPU would take an estimated 45.3 hours, scaled from 20 timed steps per model.

A further finding was made after looking at the data, and it is an association only. The price schedule, the tariff of each half-hour from the data set's own `Tariffs.xlsx`, marks the half-hours priced high and the half-hours priced low. The tariff file appears to follow UK clock time, so my pipeline moved its summer stamps one hour back to match the readings. That is my inference from when the time-of-use households change their use, as neither file says so. Over the training and validation months, the mean of the previous 7 days forecast more than the time-of-use group used in the half-hours priced high, a bias of -8.44%. It forecast less in those priced low, a bias of +7.38%. The standard tariff group's bias on the same half-hours was -0.30% and -0.52%. So the time-of-use group used less than the mean of its previous 7 days in the half-hours priced high and more in those priced low. The trial's own report expected that direction: "a negative kW response" to the high price and "a positive kW response (i.e. consumers re-scheduling laundry cycles and energy usage) to the low price" (UK Power Networks, 2015a, p. 27).

The caution is that the validation months on their own tell a mixed story. Here the gap between the groups is the time-of-use group's bias less the standard tariff group's, in percentage points ("points"). In the validation months the gap is weaker in the half-hours priced high, -6.77 points against -8.14 over both periods. It is larger in those priced low, +9.01 against +7.90. In the half-hours priced high the time-of-use group's own validation bias is -2.12%, against +4.65% for the standard tariff group. These figures rest on 114 half-hours priced high and 150 priced low. The two models, which cannot see the price signal, show gaps of the same sign on the validation months: -3.08 points for the LSTM and -4.65 for the GRU priced high, and +9.03 and +8.61

priced low. Both models forecast below the actual readings in these half-hours, with biases from +15.76% to +33.36%, so the gap between the groups is the figure to read, not the level.

In plain terms, both models learned each household's usual day well enough to forecast its next 24 hours more closely than rules of thumb such as repeating last week. The GRU, the smaller model, was slightly more accurate. Both forecasts are smoothed and run low, so they are weakest on unusual days such as a heavy Christmas Day. They are no better than a rule of thumb at saying when in the day use will peak. Each forecast is best read as a typical day for that household, not a warning of its busiest half-hours.

Limitations and Risks

The Dataset. The households are London households supplied by EDF Energy, over one year, and the trial excluded households on Economy 7 tariffs, dual fuel supply or prepayment meters (UK Power Networks, 2015a, p. 17). The pool kept only the 1,132 of 5,528 households with every 2013 reading, the test months included. So the sample was chosen on completeness that runs to 31 December, and the models are untested on households with gaps. The test months are winter and the training months mostly not. Mean use was 0.2485 kWh per half-hour on the test months against 0.1955 on the validation months. Standard theory assumes "the training set and test set are identically distributed" (Goodfellow et al., 2016, p. 109), so the test score includes a seasonal shift. The same half-hour yesterday shows it, with a median MASE of 0.9572 on the validation months and 1.1143 on the test months. The time-of-use group was under a trial intervention, and such demand "may not represent 'normal', every-day behaviour" (Haben et al., 2021, sec. 4).

The Architecture. The models read only the past week's readings, with no calendar, price or weather input, so Christmas and the price signals are invisible to them. I left weather out

from the start, although temperature is among the most common extra inputs for this kind of forecast (Haben et al., 2021, sec. 2.1.1). The published readings hold no weather, so adding it would mean bringing in a second source with its own licence check, and the project uses one data set. On Christmas Day MAC004319's MAE was 1.5298 kWh for the LSTM. The 336-step input is long for gradients to cross (Goodfellow et al., 2016, sec. 10.7), and nothing in this run shows how much of the week the models use.

The Training Setup. One value of each setting was run: two layers, 64 hidden units and a learning rate of 0.001, never tuned. The learning rate is the setting most worth tuning (Goodfellow et al., 2016, sec. 11.4.1). The L1 loss makes the forecasts medians, so both models trail the mean of the previous 7 days on RMSE (0.2366 for the LSTM and 0.2326 for the GRU, against 0.2299 kWh). Three seeds give a spread, a test MAE standard deviation of 0.0004 for the LSTM and 0.0002 for the GRU, but three is a small sample. The linear head lets forecasts go negative, down to -0.3553 kWh.

The Evaluation Approach. The test is one period of two winter months, with no evaluation on a rolling forecasting origin across the year (Hyndman & Athanasopoulos, 2021, sec. 5.10). No strong statistical model was run, such as the Holt-Winters-Taylor exponential smoothing that Haben et al. (2021, sec. 6.1) recommend, so the models are shown only to beat naive forecasts. MAE hides timing errors through the double penalty, which the peak-timing confusion matrix only partly offsets. The paired bootstrap has to resample whole households, because errors within a household are not independent, and it is my choice of method, not the comparison rule. The price-signal finding was made after looking at the data, so it is an association only, and in the validation months it is weaker in the half-hours priced high. The GPU forecasting times could not be ranked, and the CPU time is an estimate, not a measured run.

Risks of Relying on the Forecast. Anyone planning a battery's schedule or a street's evening capacity from these forecasts would under-plan the heaviest days. Both models forecast low across the test months and smooth the evening peak. They place the peak in the right 4-hour block on only 34.5% (LSTM) and 35.6% (GRU) of household-days. They err most on heavy-use days such as MAC004319's Christmas Day, which a plan most needs to get right. A user could also read the lower MAE as better peak forecasting, which Figure 4 shows it is not.

Ethical and Responsible Use

Privacy and Occupancy. Half-hourly readings reveal a household's routine. McKenna et al. (2012, sec. 2.2) found that "with a little practice it is perfectly possible to identify the general movements of residents with some confidence from half-hourly data". In UK terms, detailed smart meter data are "likely to be 'personal data'" (Department for Business, Energy & Industrial Strategy [BEIS], 2018, para. 2.1). Suppliers may access data more detailed than daily only with consent (BEIS, 2018, para. 2.8). The concern applies to the model as well as the data. It learns each household's usual day and forecasts when its use will rise and fall tomorrow. Routines show clearly here: averaged over all households, the weekday morning rise moved from 07:00 to 06:00 GMT with the March clock change. Run on live readings, such forecasts would say when a household is likely to be up, out or away. Someone could misuse that to judge when a household is empty. This report names households only by the publisher's identifiers, and the data set holds no more than readings, each household's identifier and tariff group, date and time, with the price schedule as a separate file (UK Power Networks, 2015b).

Bias Against Time-of-Use Households. The price-signal finding, made after looking at the data and an association only, and weaker in the validation months in the half-hours priced high, points to a second concern. A model that reads only past use cannot see a price signal given

a day ahead. On the validation months both models' gap between the groups has the same sign as the naive forecast's. Relative to the standard tariff group, they forecast the time-of-use group too high in the half-hours priced high and too low in those priced low. Forecasts used to plan for these households, or to judge their response to a tariff, would carry that error into decisions about them. The sample adds to it. The trial excluded households on Economy 7 tariffs, dual fuel supply or prepayment meters (UK Power Networks, 2015a, p. 17), and the evening peak differs by household type (UK Power Networks, 2015a, p. 53). My concern, which the trial report does not draw, is that errors may be larger for the households the sample leaves out.

Responsible Use. These are point forecasts with no stated uncertainty, and "point forecasts can be of almost no value without the accompanying prediction intervals" (Hyndman & Athanasopoulos, 2021, sec. 5.5). So no forecast here should drive a decision about one household on its own. Under the framework that BEIS reviewed, a supplier that ran such a model on live half-hourly readings would need the household's consent (BEIS, 2018, para. 2.8). A network operator would need consent, or data treated so that it can no longer be tied to one household (BEIS, 2018, para. 2.12).

Future Improvements

Each step answers a result or limitation above and says what it needs.

1. **Give the model the price schedule.** The prices were given a day ahead (UK Power Networks, 2015b), so the next day's half-hours priced high and low could be an input for the time-of-use group. This answers the price-signal finding and the bias concern. It needs only the data set's own price schedule as a second input feature.
2. **Add calendar inputs.** A time index can be "an extra input at each time step" (Goodfellow et al., 2016, p. 384). A day-of-week and holiday input would let the model

see that Christmas Day is not an ordinary Wednesday. It answers the Christmas failure case and needs no new data.

3. **Run a strong statistical benchmark.** Holt-Winters-Taylor exponential smoothing, as Haben et al. (2021, sec. 6.1) recommend, would show whether the models beat a fitted statistical model as well as naive forecasts. It needs only the time to fit it, as the data are already prepared.
4. **Forecast a range, not one value.** Probabilistic forecasts often suit this load better than point forecasts, because they represent the uncertainty (Haben et al., 2021, sec. 2.6). A band for each half-hour would show how far the low median forecast sits from the heavy days. It needs a new output head and a loss to train it.
5. **Score peaks directly.** Haben et al. (2021, sec. 6.1) ask peak forecasting studies to use error measures that avoid the double penalty. Selecting models on such a measure would target the peak-timing loss, where both models trail the mean of the previous 7 days. It needs that error measure in the model selection, and no new data.
6. **Evaluate on a rolling origin.** Re-fitting and testing at several points across the year (Hyndman & Athanasopoulos, 2021, sec. 5.10) would separate the models' skill from the winter shift. It needs more training runs, and one full set of six runs with the pre-flight took 21.0 minutes on the GPU.
7. **Tune the shared settings on validation.** A small search over the learning rate, the width and the depth, applied to both models alike, would test choices fixed at the outset (Goodfellow et al., 2016, sec. 11.4.1). It needs one full set of runs, about 21.0 minutes on the GPU, for each combination tried.

8. **Train on more than one year.** Training on all the available history, at least a full year and ideally several, and refitting as new readings arrive, would let the model see every season and its peaks in training. This answers the seasonal shift in the test score. It was not done here because the households took part "between November 2011 and February 2014" (UK Power Networks, 2015b, para. 1), and the trial collected readings for "the full 2013 calendar year" (UK Power Networks, 2015a, p. 12). Requiring every reading of a second year would shrink the pool, and the time-of-use group was on its tariff "throughout the 2013 calendar year period" (UK Power Networks, 2015b, para. 4), so two years of its readings would mix two tariffs. It needs several complete years of readings for the same households.

References

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1724-1734). Association for Computational Linguistics. https://doi.org/10.3115/v1/D14-1179

Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical evaluation of gated recurrent neural networks on sequence modeling* (arXiv:1412.3555). arXiv. https://doi.org/10.48550/arXiv.1412.3555

Department for Business, Energy & Industrial Strategy. (2018). *Smart Metering Implementation Programme: Review of the Data Access and Privacy Framework*. GOV.UK. https://www.gov.uk/government/publications/smart-metering-implementation-programme-review-of-the-data-access-and-privacy-framework/smart-metering-implementation-programme-review-of-the-data-access-and-privacy-framework

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics, 7*(1), 1-26. https://doi.org/10.1214/aos/1176344552

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org

GOV.UK. (2025, November 24). *When do the clocks change?* Retrieved October 11, 2026, from https://www.gov.uk/when-do-the-clocks-change

Haben, S., Arora, S., Giasemidis, G., Voss, M., & Vukadinović Greetham, D. (2021). Review of low voltage load forecasting: Methods, applications, and recommendations. *Applied Energy, 304*, Article 117798. https://doi.org/10.1016/j.apenergy.2021.117798

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735-1780. https://doi.org/10.1162/neco.1997.9.8.1735

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. Retrieved October 11, 2026, from https://otexts.com/fpp3/

Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting, 22*(4), 679-688. https://doi.org/10.1016/j.ijforecast.2006.03.001

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An introduction to statistical learning: With applications in Python*. Springer. https://doi.org/10.1007/978-3-031-38747-0

Kingma, D. P., & Ba, J. (2015, May 7-9). *Adam: A method for stochastic optimization* [Paper presentation]. 3rd International Conference on Learning Representations, San Diego, CA, United States. https://doi.org/10.48550/arXiv.1412.6980

McKenna, E., Richardson, I., & Thomson, M. (2012). Smart meter data: Balancing consumer privacy concerns with legitimate applications. *Energy Policy, 41*, 807-814. https://doi.org/10.1016/j.enpol.2011.11.049

Prechelt, L. (1998). Early stopping: But when? In G. B. Orr & K.-R. Müller (Eds.), *Neural networks: Tricks of the trade* (pp. 55-69). Springer. https://doi.org/10.1007/3-540-49430-8_3

PyTorch. (n.d.-a). *torch.nn.GRU* (Version 2.14). Retrieved October 11, 2026, from https://docs.pytorch.org/docs/2.14/generated/torch.nn.GRU.html

PyTorch. (n.d.-b). *torch.nn.L1Loss* (Version 2.14). Retrieved October 11, 2026, from https://docs.pytorch.org/docs/2.14/generated/torch.nn.L1Loss.html

PyTorch. (n.d.-c). *torch.nn.LSTM* (Version 2.14). Retrieved October 11, 2026, from https://docs.pytorch.org/docs/2.14/generated/torch.nn.LSTM.html

UK Power Networks. (2015a). *Low Carbon London project closedown report* (Version 1.0). https://d1oyzg0jo3ox9g.cloudfront.net/app/uploads/2023/10/Project-Closedown-Report-March-2015.pdf

UK Power Networks. (2015b). *SmartMeter energy consumption data in London households* [Data set]. London Datastore. https://data.london.gov.uk/dataset/smartmeter-energy-use-data-in-london-households

Van den Broeck, J., Argeseanu Cunningham, S., Eeckels, R., & Herbst, K. (2005). Data cleaning: Detecting, diagnosing, and editing data abnormalities. *PLoS Medicine, 2*(10), Article e267. https://doi.org/10.1371/journal.pmed.0020267